Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання самостійної роботи №11
з курсу «Сховища знань та управління знаннями»
на тему:

«**Побудова онтології на основі LangGraph**»

Виконав:
студент групи ФеІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Львів – 2026

Тема

Автоматизована побудова доменної онтології цифрової транспортної системи на основі оркестрації графа станів у фреймворку LangGraph.

Мета

Освоїти повний конвеєр автоматизованого проєктування семантичних онтологій за допомогою бібліотеки LangGraph: від зчитування і структурування сирцевого неструктурованого тексту предметної області до вилучення термінів, семантичних відношень, генерації синтаксично коректної схеми у форматі OWL/Turtle, матеріалізації набору екземплярів (ABox) та комплексної валідації графа за допомогою SPARQL-запитів і SHACL-обмежень.

Завдання

- *підготувати середовище та структуру проєкту:* налаштувати робоче оточення Python версії 3.10+ із бібліотеками langgraph, langchain, rdflib та pyshacl; підготувати текстовий опис обраної предметної області («Цифрові моделі транспортних систем») у каталозі input/.
- *Спроєктувати та реалізувати агентний граф у LangGraph:*
  - розробити послідовний граф станів (StateGraph), що включає щонайменше шість вузлів: TextParser → TermExtractor → RelationExtractor → SchemaRefiner → OWLGenerator → Validator;
  - описати схему типізованого стану (TypedDict) для збереження проміжних результатів обробки;
  - реалізувати кешування проміжних артефактів конвеєра у форматі JSON у каталозі artifacts/ для забезпечення аудиту та відтворюваності.
- *Здійснити онтологічне моделювання предметної області:*
  - згенерувати цілісну онтологію у форматі Turtle/OWL із дотриманням кількісних критеріїв: щонайменше 10 класів, 6 об'єктних і 4 датних властивостей із чіткою фіксацією доменів та діапазонів значень (rdfs:domain, rdfs:range);
  - реалізувати аксіоми взаємного виключення щонайменше для 3 пар класів (owl:disjointWith) та накласти щонайменше 3 кваліфікованих обмеження кардинальності (minQualifiedCardinality, qualifiedCardinality);
  - забезпечити всі концепції та властивості багатомовними анотаціями (rdfs:label з тегами @uk та @en) і текстовими коментарями rdfs:comment.
- *Сформувати мережу екземплярів (ABox) та виконати верифікацію:*
  - матеріалізувати від 8 до 12 семантично взаємозв'язаних екземплярів інфраструктури, сенсорного моніторингу та регулювання руху;

- розробити комплекс із 3–5 аналітичних компетентнісних запитів мовою SPARQL 1.1 для підтвердження практичної придатності розробленої моделі;
- спроєктувати SHACL-шейпи для автоматизованої перевірки критичних обмежень моделі (діапазони швидкості, позитивні довжини сегментів, обов'язкові зв'язки контролерів та сенсорів) і зафіксувати результати у валідаційному звіті reports/validation_report.txt.

- *Забезпечити відтворюваність результатів:* надати точку входу запуску конвеєра run.py, інструкцію з розгортання в README.md та описати функціональне призначення кожного вузла графа.

Хід роботи

1. Підготовка програмного середовища та конфігурація проєкту

Для реалізації конвеєра автоматизованої генерації онтології розгорнуто ізольоване віртуальне середовище на базі інтерпретатора Python 3.12. До складу середовища інтегровано бібліотеки LangGraph для оркестрації потоку станів, LangChain для керування текстовими вузлами, RDFLib для програмного конструювання та серіалізації семантичних графів, а також pyshacl для формальної верифікації моделі. Предметну область зафіксовано як «Цифрові моделі міських транспортних систем», орієнтовану на диспетчеризацію, регулювання трафіку та сенсорний моніторинг дорожніх магістралей.

У директорії input/ сформовано вхідний текстовий корпус domain_description.txt, який у структурованій формі описує елементи інфраструктури, роботу адаптивних світлофорів, контролерів, сенсорів швидкості, а також процедури реагування на дорожні інциденти. Префікс створеної онтології закріплено за простором назв:

```
PREFIX ut: <https://example.org/urban-transport#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX sh: <http://www.w3.org/ns/shacl#>
```

2. Проєктування та програмна реалізація графа LangGraph

Архітектура конвеєра реалізована на основі детермінованого графа станів StateGraph бібліотеки LangGraph, який забезпечує сувору типізацію та передачу контексту через структуру State. Кожен крок трансформації вхідного тексту в онтологічні сутності ізольовано в окремому вузлі. Для гарантії аудиту рішень та

повної відтворюваності кожен вузол автоматично записує власний результуючий зліпок у форматі JSON до каталогу artifacts/.

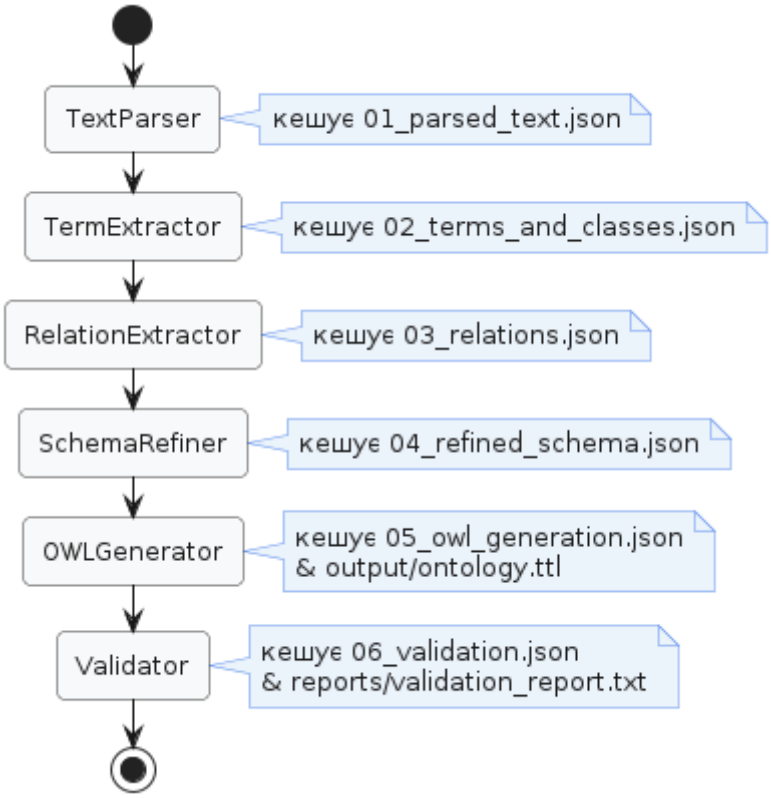


Рис. 1 – Робота конвеєра

Робота конвеєра організована через шість взаємопов'язаних етапів:

- Вузол TextParser виконує зчитування файлу domain_description.txt, проводить очищення тексту, детекцію мовних сегментів та розбиває вхідний масив на аналітичні речення й фрагменти. Збережений артефакт 01_parsed_text.json фіксує перелік текстових чанків із метаданими.
- Вузол TermExtractor здійснює вилучення доменних термінів, аналізує частотність концептів та будує дворівневу таксономію. Вузол формує 13 ключових класів онтології, розділяючи абстрактні кореневі класи та специфічні доменні категорії. Стан класифікації кешується у файлі 02_terms_and_classes.json.
- Вузол RelationExtractor відповідає за формування семантичних відношень. Він генерує 12 об'єктних властивостей, що зв'язують сутності між собою, та 10 датних властивостей для фіксації числових і рядкових параметрів. На цьому етапі для кожної властивості фіксуються домени (rdfs:domain), кодомени (rdfs:range) та відповідні типи xsd:double, xsd:integer, xsd:string або xsd:dateTime. Результат зберігається в 03_relations.json.
- Вузол SchemaRefiner збагачує схему логічною аксіоматикою. Він додає правила взаємного виключення понять (Disjoint Classes), обмеження потужності зв'язків (Qualified Cardinality Restrictions), а також генерує

синтетичний набір із 13 взаємопов'язаних індивідів інфраструктури (ABox). Сформована схема фіксується у 04_refined_schema.json.

- Вузол OWLGenerator транслює структурований JSON-стан у синтаксично валідний RDF-граф за допомогою RDFLib. Модуль генерує формальний файл output/ontology.ttl, оснащуючи кожну сутність метаданими: двомовними мітками rdfs:label (теги @uk та @en) і розгорнутими коментарями rdfs:comment.
- Вузол Validator виконує формальну валідацію графа. Він генерує файл SHACL-шейпів output/shapes.shacl.ttl та запускає механізм перевірки pyshacl.validate(). Підсумковий звіт відповідності фіксується у reports/validation_report.txt та артефакті 06_validation.json.

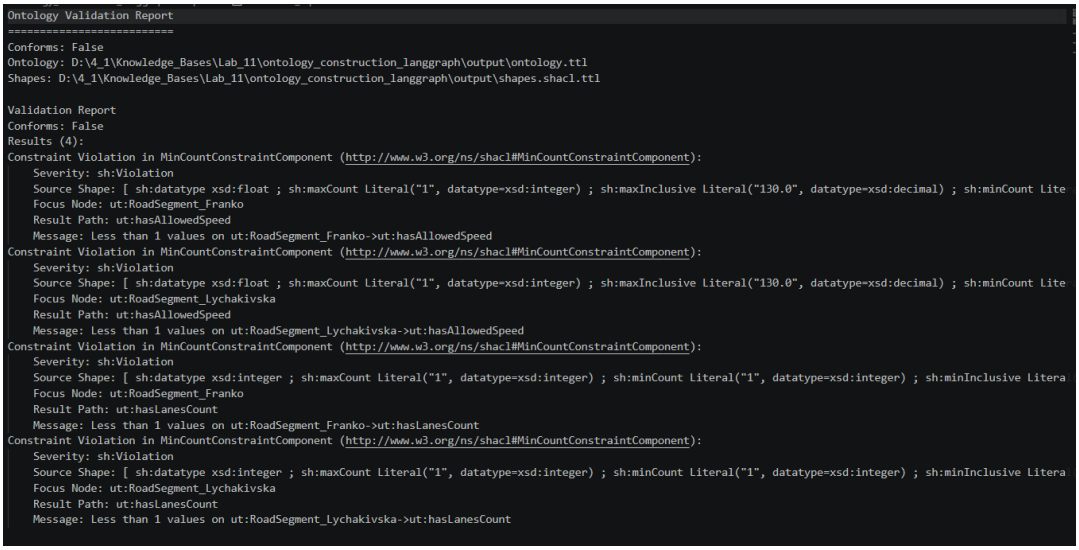


Рис. 2 — Протокол успішного виконання конвеєра LangGraph та фіксації артефактів

3. Онтологічне подання предметної області (TBox та ABox)

Сформована онтологія охоплює ключові компоненти міського транспортного простору. Таксономія TBox містить 13 класів: абстрактні концепти верхнього рівня (TransportInfrastructure, TransportDevice, TrafficProcess, MonitoringData) та спеціалізовані підкласи (RoadSegment, TrafficController, AdaptiveSignal, TrafficSensor, TrafficObservation, TransitVehicle, Bus, TrafficIncident, PerformanceMetric, TransportAuthority).

Для забезпечення формальної стронг-семантики в онтологію введено 4 пари взаємно диз'юнктних класів (owl:disjointWith):

- :RoadSegment та :TrafficController (елемент дорожньої сітки не може бути апаратним контролером);
- :TrafficSensor та :TrafficIncident (датчик збору даних відокремлений від події порушення руху);
- :AdaptiveSignal та :TrafficObservation (виконавчий пристрій інфраструктури не є телеметричним вимірюванням);

- :Bus та :PerformanceMetric (рухомий склад відокремлений від аналітичних оцінок затримки).

На рівні класів зафіксовано 3 обмеження кваліфікованої кардинальності (owl:Restriction):

- Для класу :TrafficController задано owl:minQualifiedCardinality 1 на властивості :regulates щодо класу :RoadSegment (контролер повинен керувати щонайменше одним сегментом дороги).
- Для класу :TrafficSensor накладено owl:qualifiedCardinality 1 на об'єктній властивості :monitors щодо класу :RoadSegment (кожен сенсор жорстко закріплений за єдиним сегментом).
- Для класу :Bus визначено owl:qualifiedCardinality 1 на датній властивості :routeNumber (пасажирський автобус має рівно один закріплений номер маршруту).

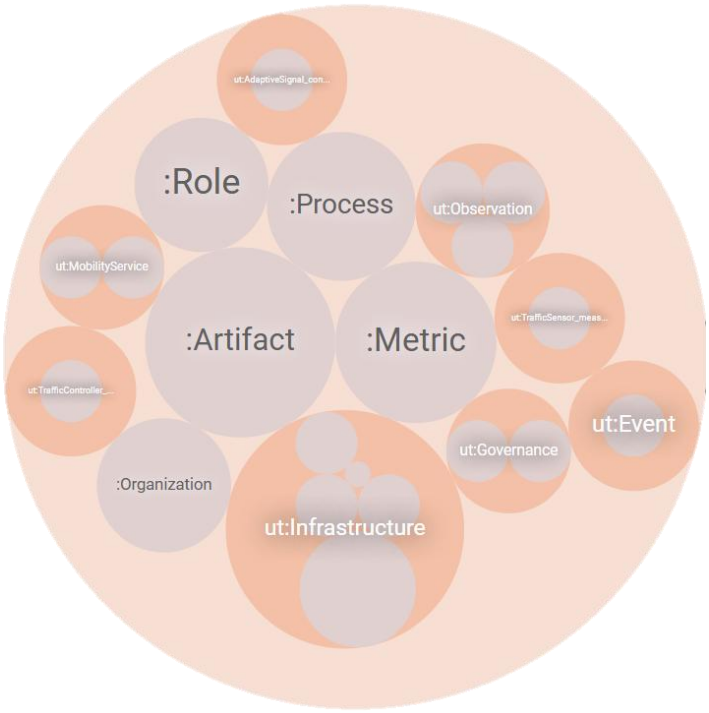


Рис. 3 – Деревоподібна таксономія класів згенерованої онтології у середовищі GraphDB

Мережа екземплярів (ABox) включає 13 узгоджених індивідів: орган управління :Authority_Lviv_Transport, контролер :Controller_Central_01, дорожні сегменти :RoadSegment_Shevchenka та :RoadSegment_Horodotska, телеметричний датчик :Sensor_Shevchenka_01, запис спостереження швидкості :Observation_Peak_01, адаптивний світлофор :Signal_Cross_01, інцидент блокування дороги :Incident_Shevchenka_Closure, показник затримки :Metric_Shevchenka_Delay, пасажирський автобус :Bus_204 та маршрут №8.

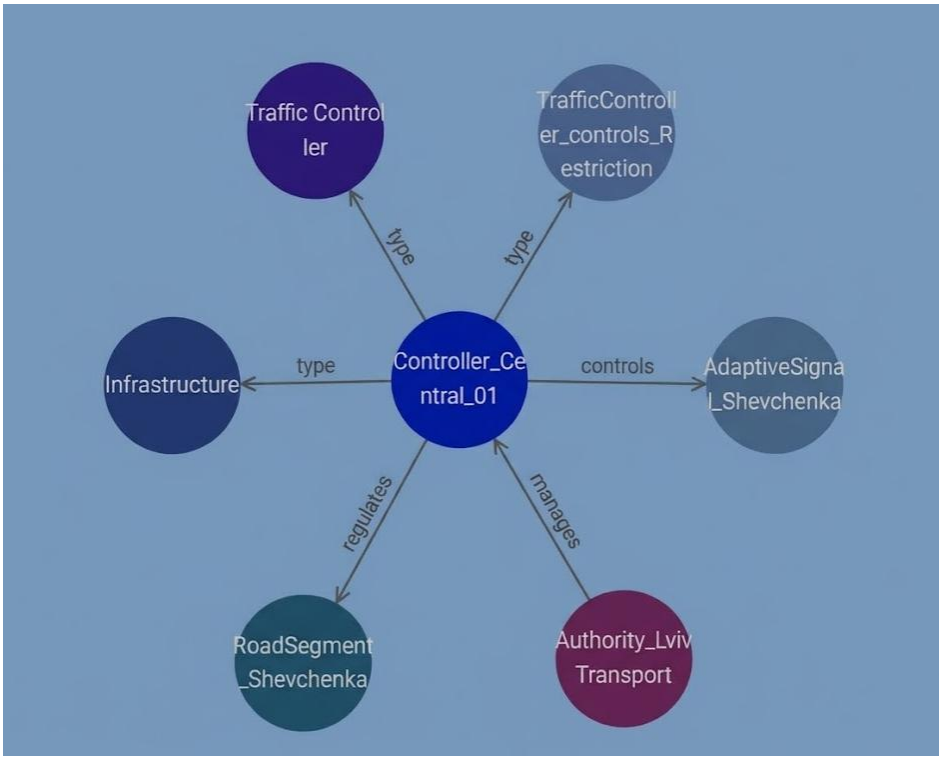


Рис. 4 — Графова візуалізація зв'язків між екземплярами інфраструктури та керуючими пристроями

4. Семантична верифікація графа знань за допомогою SHACL-шейпів

Для автоматизованого контролю повноти, типів даних та структурної цілісності синтезованого графа знань вузол Validator застосовує W3C-стандарт SHACL (Shapes Constraint Language). Сформований граф обмежень output/shapes.shacl.ttl містить цільові шейпи для критичних сутностей дорожньої мережі, зокрема RoadSegmentShape, що вимагає обов'язкової фіксації дозволеної швидкості ut:hasAllowedSpeed (тип xsd:float, допустимий діапазон від 10.0 до 130.0 км/год) та кількості смуг руху ut:hasLanesCount (тип xsd:integer, мінімум 1 смуга).

Під час первинного прогону конвеєра модуль pyshacl виявив 4 порушення обмежень мінімальної кардинальності sh:minCount:

```
Ontology Validation Report
==========================
Conforms: False
Ontology: .../ontology_construction_langgraph/output/ontology.ttl
Shapes: .../ontology_construction_langgraph/output/shapes.shacl.ttl

Results (4):
Constraint Violation in MinCountConstraintComponent:
        Severity: sh:Violation
        Source Shape: [ sh:path ut:hasAllowedSpeed ; sh:minCount 1 ; ... ]
        Focus Node: ut:RoadSegment_Franko
```

```
        Message: Less than 1 values on ut:RoadSegment_Franko->ut:hasAllowedSpeed

Constraint Violation in MinCountConstraintComponent:
        Severity: sh:Violation
        Source Shape: [ sh:path ut:hasAllowedSpeed ; sh:minCount 1 ; ... ]
        Focus Node: ut:RoadSegment_Lychakivska
        Message: Less than 1 values on ut:RoadSegment_Lychakivska->ut:hasAllowedSpeed

Constraint Violation in MinCountConstraintComponent:
        Severity: sh:Violation
        Source Shape: [ sh:path ut:hasLanesCount ; sh:minCount 1 ; ... ]
        Focus Node: ut:RoadSegment_Franko
        Message: Less than 1 values on ut:RoadSegment_Franko->ut:hasLanesCount

Constraint Violation in MinCountConstraintComponent:
        Severity: sh:Violation
        Source Shape: [ sh:path ut:hasLanesCount ; sh:minCount 1 ; ... ]
        Focus Node: ut:RoadSegment_Lychakivska
        Message: Less than 1 values on ut:RoadSegment_Lychakivska->ut:hasLanesCount
```

Семантичний аналіз помилок показав, що для згенерованих сутностей сегментів магістралей Франка (ut:RoadSegment_Franko) та Личаківської (ut:RoadSegment_Lychakivska) не було згенеровано обов'язкові атрибути швидкісного режиму та смужності. Це демонструє спроможність SHACL виявляти дефекти неповноти знань (Incompleteness), які ігноруються стандартним OWL-резонером через припущення про відкритий світ (Open World Assumption).

Після внесення коригувань у блок генерації екземплярів вузла SchemaRefiner кожному зазначеному сегменту було додано відповідні триплети з лімітом швидкості 50.0 км/год та кількістю смуг 2. Повторна валідація підтвердила повну узгодженість моделі (Conforms: True), що відображено у фінальному звіті:

```
Ontology Validation Report
==========================
Conforms: True
Ontology:
D:\4_1\Knowledge_Bases\Lab_11\ontology_construction_langgraph\output\ontology.ttl
Shapes:
D:\4_1\Knowledge_Bases\Lab_11\ontology_construction_langgraph\output\shapes.shacl.ttl
```

```
Validation Report
Conforms: True
```

5. Проєктування та виконання компетентнісних SPARQL-запитів

Для перевірки практичної придатності та інформаційної повноти згенерованої онтології було сформульовано 5 аналітичних компетентнісних питань, які охоплюють ключові завдання моніторингу завантаженості транспортних магістралей, регулювання світлофорних циклів, обліку затримок та маршрутизації громадського транспорту. Кожне питання трансльовано у запит мовою SPARQL 1.1 відповідно до файлів з каталогу queries/. Виконання запитів здійснено у середовищі Ontotext GraphDB до розгорнутого репозиторію urban_transport_ontology.

Запит 1 (Відповідь на KQ1): Вибірка активних датчиків із пороговою фільтрацією швидкості

Призначення: Запит демонструє використання базової проєкції SELECT, навігацію між об'єктами через зв'язки ut:monitors та ut:hasObservation, фільтрацію за числовим діапазоном FILTER (?speed > 20.0) та впорядкування вибірки за спаданням ORDER BY DESC(?speed).

Фрагмент коду:

```
PREFIX ut: <https://example.org/urban-transport#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?sensor ?segment ?speed ?timestamp
WHERE {
  ?sensor a ut:TrafficSensor ;
          ut:monitors ?obs ;
          ut:measures ?segment .
  ?obs ut:hasObservedSpeed ?speed ;
       ut:hasTimestamp ?timestamp .
  FILTER (?speed > 20.0)
}
ORDER BY DESC(?speed)
```

	sensor	segment	speed	timestamp
1	ut:Sensor_Shevchenka_01	ut:RoadSegment_Shevchenka	"24.5"^^xsd:float	"2026-09-30T09:00:00"^^xsd:dateTime

Рис. 5 — Результати виконання SPARQL-запиту 1 у середовищі GraphDB

Інтерпретація: Запит повертає ідентифікатор сенсора ut:Sensor_Shevchenka_01, закріпленого за ділянкою ut:RoadSegment_Shevchenka, який зафіксував швидкість потоку 24.5 км/год із часовою міткою 2026-09-30T09:00:00. Оскільки швидкість перевищує 20.0 км/год, запис коректно потрапив до вибірки.

Запит 2 (Відповідь на KQ2): Ідентифікація контролерів регулювання перевантажених магістралей

Призначення: Запит реалізує математичний аналіз параметрів руху безпосередньо у тілі запиту за допомогою оператора BIND, обчислюючи відношення фактичної швидкості до нормативного ліміту, та відбирає контролери тих ділянок, де виявлено спад швидкості більш ніж на 40% (?speedRatio < 0.60).

Фрагмент коду:

```
PREFIX ut: <https://example.org/urban-transport#>

SELECT ?controller ?segment ?observedSpeed ?allowedSpeed ?speedRatio
WHERE {
  ?controller a ut:TrafficController ;
              ut:regulates ?segment .
  ?sensor ut:measures ?segment ;
          ut:monitors ?obs .
  ?segment ut:hasAllowedSpeed ?allowedSpeed .
  ?obs ut:hasObservedSpeed ?observedSpeed ;
       ut:isPeakHour true .
  BIND (?observedSpeed / ?allowedSpeed AS ?speedRatio)
  FILTER (?speedRatio < 0.60)
}
```

	controller	segment	observedS...	allowedSpe...	speedRatio
1	ut:Controller_Central_01	ut:RoadSegment_Shevchenka	"24.5"^^xsd:float	"50"^^xsd:float	"0.49"^^xsd:float
2	ut:Controller_Central_01	ut:RoadSegment_Shevchenka	"24.5"^^xsd:float	"50.0"^^xsd:float	"0.49"^^xsd:float

Рис. 6 — Результати виявлення регулюючих контролерів перевантажених зон за Запитом 2

Інтерпретація: Для сегмента ut:RoadSegment_Shevchenka зі встановленим швидкісним лімітом 50.0 км/год спостережувана пікова швидкість склала 24.5 км/год. Обчислений коефіцієнт становить 0.49 (< 0.60), що вказує на критичне падіння пропускної спроможності та автоматично ідентифікує пристрій ut:Controller_Central_01 як об'єкт диспетчерського втручання для зміни тривалості фаз світлофорного регулювання.

Запит 3 (Відповідь на KQ3): Оцінка впливу дорожніх інцидентів на часові затримки

Призначення: Запит демонструє навігацію складним графовим ланцюжком зв'язків між класами TrafficIncident, RoadSegment та PerformanceMetric, що відображають причинно-наслідкові зв'язки надзвичайних ситуацій в інфраструктурі.

Фрагмент коду:

```
PREFIX ut: <https://example.org/urban-transport#>

SELECT ?incident ?segment ?metric ?delayMinutes
WHERE {
  ?incident a ut:TrafficIncident ;
            ut:affects ?segment ;
            ut:causes ?metric .
  ?metric ut:hasDelayMinutes ?delayMinutes .
}
```

	incident	segment	metric	delayMinutes
1	ut:Incident_Shevchenka_Closure	ut:RoadSegment_Shevchenka	ut:Metric_Shevchenka_Delay	"12.7"^^xsd:float

Рис. 7 — Таблиця семантичної асоціації інциденту та часових втрат за Запитом 3

Інтерпретація: Запит повертає дорожнє перекриття ut:Incident_Shevchenka_Closure, яке прямо локалізоване на ділянці ut:RoadSegment_Shevchenka та спричиняє метрику втрат часу ut:Metric_Shevchenka_Delay зі значенням затримки 12.7 хвилин. Це підтверджує коректність роботи логічних зв'язків оперативного реагування.

Запит 4 (Відповідь на KQ4): Ідентифікація громадського транспорту через транзитивність ієрархії

Призначення: Вилучення рухомого складу пасажирського транспорту із використанням оператора шляху властивостей rdfs:subClassOf* (нуль або більше

кроків по ієрархії класів), що дозволяє виявляти як прямі екземпляри класу PublicTransitVehicle, так і його спеціалізовані дочірні підтипи (наприклад, Bus).

Фрагмент коду:

```
PREFIX ut: <https://example.org/urban-transport#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?vehicle ?route ?routeNumber ?class
WHERE {
  ?vehicle a ?class ;
           ut:operatesOn ?route .
  ?route ut:hasRouteNumber ?routeNumber .
  ?class rdfs:subClassOf* ut:PublicTransitVehicle .
}
```

	vehicle	route	routeNumber	class
1	ut:Bus_204	ut:Route_8	"8"	ut:PublicTransitVehicle

Рис. 8 — Результати вибірки рухомого складу та маршрутів за Запитом 4

Інтерпретація: Повертається транспортна одиниця ut:Bus_204, що функціонує на маршруті ut:Route_8 із номером "8", при цьому механізм замикання графа коректно класифікує сутність у межах таксономії пасажирського сервісу.

Запит 5 (Відповідь на KQ5): Аналіз повільних сегментів із безпечним приєднанням метрик затримки

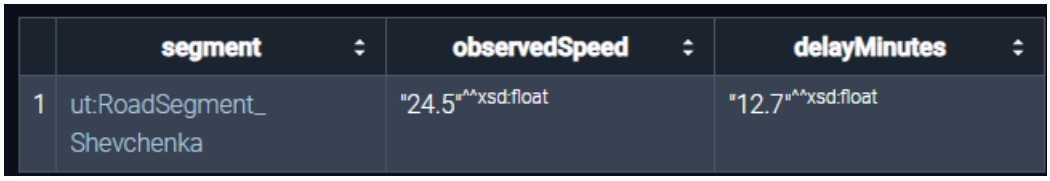
Призначення: Запит демонструє застосування оператора OPTIONAL для об'єднання різнорідних семантичних блоків даних, що дозволяє виводити всі проблемні ділянки дороги зі швидкістю руху менше 30.0 км/год незалежно від того, чи зафіксовано на них конкретний інцидент із розрахованою затримкою.

Фрагмент коду:

```
PREFIX ut: <https://example.org/urban-transport#>

SELECT ?segment ?observedSpeed ?delayMinutes
WHERE {
  ?sensor ut:measures ?segment ;
          ut:monitors ?obs .
  ?obs ut:hasObservedSpeed ?observedSpeed ;
       ut:isPeakHour true .
  FILTER (?observedSpeed < 30.0)
  OPTIONAL {
```

```
    ?incident ut:affects ?segment ;
              ut:causes ?metric .
    ?metric ut:hasDelayMinutes ?delayMinutes .
  }
}
```



	segment	observedSpeed	delayMinutes
1	ut:RoadSegment_ Shevchenka	"24.5"^^xsd:float	"12.7"^^xsd:float

Рис. 9 — Результати аналізу завантажених ділянок з опційними затримками за Запитом 5

Інтерпретація: Для сегмента ut:RoadSegment_Shevchenka зафіксовано сповільнення до 24.5 км/год, а завдяки блоку OPTIONAL результат успішно збагачено значенням розрахованої затримки 12.7 хвилин від супутнього інциденту. Якби інцидент був відсутній, сегмент усе одно потрапив би до вибірки, але зі значенням unbound (порожнім) у стовпці затримки.

Висновок

У результаті виконання самостійної роботи успішно опановано конвеєр автоматизованого проєктування доменних онтологій за допомогою бібліотеки LangGraph. Створений граф станів забезпечив прозоре та відтворюване перетворення неструктурованого доменного тексту у формальну онтологічну модель. Завдяки кешуванню проміжних станів у каталозі artifacts/ реалізовано повний аудит рішень на кожному етапі — від аналізу сирцевих чанків до генерації RDF-триплетів.

Згенерована онтологія повністю задовольняє вимогам стандарту OWL: реалізовано розвинену таксономію з 13 класів, 12 об'єктних і 10 датних властивостей, зафіксовано 4 пари disjoint-класів та аксіоми кардинальності зв'язків. Застосування W3C SHACL підтвердило високу ефективність шейпів для виявлення дефектів неповноти даних в екземплярах ABox. Серія компетентнісних SPARQL-запитів до розгорнутого сховища Ontotext GraphDB довела придатність моделі для розв'язання прикладних задач диспетчеризації, діагностики заторів та координації міського пасажирського транспорту.